

Comprehensive Report on Transformers Notebooks

Par: Abdelhamid SAIDI
Repository: Deep-Learning-FST/transformers
Total Notebooks: 11

Executive Summary

This collection of notebooks provides a comprehensive guide to Natural Language Processing (NLP) with Transformers, published from the book “NLP with Transformers”. The notebooks progress from foundational concepts to advanced applications and production-level optimizations, covering the complete lifecycle of transformer-based NLP projects.

Notebook Breakdown

Notebook 1: Hello Transformers - Introduction

File: 01_introduction.ipynb

Main Topics: - Transformer timeline and evolution - Encoder-decoder architecture fundamentals - Attention mechanisms and self-attention - Transfer learning in NLP (ULMFiT) - Hugging Face ecosystem overview

Key Sections: 1. **The Encoder-Decoder Framework** - Visual explanation of RNN architectures and their evolution to transformers 2. **Attention Mechanisms** - Introduction to attention and how it improves seq2seq models 3. **Transfer Learning in NLP** - Discusses ULMFiT and how pre-training improves downstream task performance 4. **Hugging Face Ecosystem** - Overview of key libraries: Transformers, Datasets, Tokenizers, Accelerate

Practical Applications Demonstrated:

Text Classification Example:

```
from transformers import pipeline

classifier = pipeline("text-classification")
text = """Dear Amazon, last week I ordered an Optimus Prime action figure from your online store in Germany. Unfortunately, when I opened the package, I discovered to my horror that I had been sent an action figure of Megatron instead!"""

outputs = classifier(text)
pd.DataFrame(outputs) # [{'label': 'NEGATIVE', 'score': 0.999}]
```

- **Named Entity Recognition (NER)** - Extracting entities from text

```
ner_tagger = pipeline("ner", aggregation_strategy="simple")
outputs = ner_tagger(text)
pd.DataFrame(outputs) # Returns entity locations and types
```

- **Question Answering** - Finding answers in context

```
reader = pipeline("question-answering")
question = "What does the customer want?"
outputs = reader(question=question, context=text)
pd.DataFrame([outputs]) # Returns answer and span position
```

- **Summarization** - Generating concise summaries

```
summarizer = pipeline("summarization")
outputs = summarizer(text, max_length=45, clean_up_tokenization_spaces=True)
print(outputs[0]['summary_text'])
```

- **Translation** - Machine translation (English to German)

```
translator = pipeline("translation_en_to_de",
                      model="Helsinki-NLP/opus-mt-en-de")
outputs = translator(text, clean_up_tokenization_spaces=True, min_length=100)
print(outputs[0]['translation_text'])
```

- **Text Generation** - Generating customer service responses

```
from transformers import set_seed
set_seed(42)
generator = pipeline("text-generation")
response = "Dear Bumblebee, I am sorry to hear that your order was mixed up."
prompt = text + "\n\nCustomer service response:\n" + response
outputs = generator(prompt, max_length=200)
print(outputs[0]['generated_text'])
```

Key Learnings: - Transformers replaced RNNs with attention mechanisms
 - Pre-trained models can be fine-tuned for various tasks - Hugging Face Hub provides access to thousands of pre-trained models - Pipelines enable quick prototyping without deep architectural knowledge

Technologies Used: Transformers library, PyTorch, Hugging Face Hub

Notebook 2: Text Classification

File: 02_classification.ipynb

Main Topics: - Text classification fundamentals and use cases - Emotion detection from Twitter data (6 emotions: anger, disgust, fear, joy, sadness,

surprise) - Hugging Face Datasets library - DistilBERT for efficient classification
- Data exploration and class distribution analysis

Key Sections: 1. **The Emotion Dataset** - 6-class emotion classification on Twitter data 2. **Loading Data with Hugging Face Datasets** - Using `load_dataset()` function - Exploring dataset structure and features - Converting datasets to pandas DataFrames for analysis 3. **Data Exploration** - Class distribution analysis - Dataset sanity checks - Data visualization

Dataset Details: - Source: Contextualized Affect Representations for Emotion Recognition (CARER) - Classes: anger, disgust, fear, joy, sadness, surprise - Splits: train, validation, test - Format: Tweet text + emotion label

Code Examples - Data Loading:

```
from datasets import load_dataset, list_datasets

# List all available datasets
all_datasets = list_datasets()
print(f"There are {len(all_datasets)} datasets available on the Hub")

# Load emotion dataset
emotions = load_dataset("emotion")
print(emotions) # DatasetDict with 'train', 'validation', 'test' splits

# Explore structure
train_ds = emotions["train"]
print(f"Dataset size: {len(train_ds)}") # 16000
print(f"Column names: {train_ds.column_names}") # ['text', 'label']
print(f"Features: {train_ds.features}")

# Access examples
print(train_ds[0]) # Single example as dict
print(train_ds[:5]) # Multiple examples
print(train_ds["text"][:5]) # All text in first 5 rows
```

Alternative Data Loading:

```
# Load from local CSV
emotions_local = load_dataset("csv", data_files="train.txt",
                              sep=";", names=["text", "label"])

# Load from remote URL
dataset_url = "https://huggingface.co/datasets/transformersbook/emotion-train-split/raw/main"
emotions_remote = load_dataset("csv", data_files=dataset_url,
                              sep=";", names=["text", "label"])
```

Convert to Pandas for Analysis:

```
emotions.set_format(type="pandas")
```

```
df = emotions["train"][:]
print(df.head())

# Add human-readable labels
def label_int2str(row):
    return emotions["train"].features["label"].int2str(row)

df["label_name"] = df["label"].apply(label_int2str)
print(df.head()) # Now shows actual emotion names
```

Key Learnings: - Dataset class behaves like Python lists/dictionaries - Arrow format provides memory efficiency - ClassLabel feature automatically maps integers to label names - Data exploration is critical before training

Technologies Used: Datasets, Apache Arrow, Pandas, DistilBERT

Notebook 3: Transformer Anatomy

File: 03_transformer-anatomy.ipynb

Main Topics: - Deep dive into transformer architecture - Building blocks: embeddings, self-attention, feed-forward networks - Scaled dot-product attention mechanism - Multi-headed attention implementation - Implementing transformer components from scratch

Key Sections: 1. **The Transformer Architecture** - Encoder-decoder structure with visual diagrams - Layer normalization and residual connections

2. **The Encoder**
 - Self-attention layer
 - Feed-forward network
 - Layer composition
3. **Self-Attention Mechanism**
 - From contextualized embeddings concept
 - **Scaled Dot-Product Attention:**
 - Query, Key, Value projections
 - Attention scores = $\text{softmax}(Q \cdot K^T / d_k)$
 - Output = attention_weights · V
 - Mathematical formulation and implementation
4. **Multi-Headed Attention**
 - Multiple parallel attention heads
 - Concatenation of head outputs
 - Linear projection to final output
 - PyTorch implementation with `nn.ModuleList`
5. **Feed-Forward Layer**
 - Two linear transformations with GELU activation
 - Dropout for regularization

- Expands to `intermediate_size` then back to `hidden_size`

Code Examples - Scaled Dot-Product Attention:

```
import torch
from math import sqrt
import torch.nn.functional as F

# Create embeddings from input tokens
from transformers import AutoTokenizer, AutoConfig
from torch import nn

model_ckpt = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_ckpt)
config = AutoConfig.from_pretrained(model_ckpt)

text = "time flies like an arrow"
inputs = tokenizer(text, return_tensors="pt", add_special_tokens=False)
input_ids = inputs.input_ids

# Create token embeddings
token_emb = nn.Embedding(config.vocab_size, config.hidden_size)
inputs_embeds = token_emb(input_ids) # Shape: [1, 5, 768]

# Compute scaled dot-product attention
query = key = value = inputs_embeds
dim_k = key.size(-1)

# Attention scores = softmax(Q·K^T / d_k)
scores = torch.bmm(query, key.transpose(1, 2)) / sqrt(dim_k) # [1, 5, 5]
weights = F.softmax(scores, dim=-1) # Normalize to [0, 1]

# Attention output = weights · V
attn_outputs = torch.bmm(weights, value) # [1, 5, 768]
print(f"Attention output shape: {attn_outputs.shape}")
```

Scaled Dot-Product Attention Function:

```
def scaled_dot_product_attention(query, key, value):
    dim_k = query.size(-1)
    scores = torch.bmm(query, key.transpose(1, 2)) / sqrt(dim_k)
    weights = F.softmax(scores, dim=-1)
    return torch.bmm(weights, value)

# Usage
attn_out = scaled_dot_product_attention(query, key, value)
```

Multi-Headed Attention Implementation:

```

class AttentionHead(nn.Module):
    def __init__(self, embed_dim, head_dim):
        super().__init__()
        self.q = nn.Linear(embed_dim, head_dim)
        self.k = nn.Linear(embed_dim, head_dim)
        self.v = nn.Linear(embed_dim, head_dim)

    def forward(self, hidden_state):
        attn_outputs = scaled_dot_product_attention(
            self.q(hidden_state), self.k(hidden_state), self.v(hidden_state))
        return attn_outputs

class MultiHeadAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        embed_dim = config.hidden_size
        num_heads = config.num_attention_heads
        head_dim = embed_dim // num_heads

        self.heads = nn.ModuleList(
            [AttentionHead(embed_dim, head_dim) for _ in range(num_heads)]
        )
        self.output_linear = nn.Linear(embed_dim, embed_dim)

    def forward(self, hidden_state):
        # Apply all attention heads in parallel
        x = torch.cat([h(hidden_state) for h in self.heads], dim=-1)
        # Final linear projection
        x = self.output_linear(x)
        return x

# Usage
multihead_attn = MultiHeadAttention(config)
attn_output = multihead_attn(inputs_embeds) # [1, 5, 768]

```

Feed-Forward Network:

```

class FeedForward(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.linear_1 = nn.Linear(config.hidden_size, config.intermediate_size)
        self.linear_2 = nn.Linear(config.intermediate_size, config.hidden_size)
        self.gelu = nn.GELU()
        self.dropout = nn.Dropout(config.hidden_dropout_prob)

    def forward(self, x):
        x = self.linear_1(x) # Expand: 768 → 3072

```

```

        x = self.gelu(x)          # Non-linearity
        x = self.linear_2(x)      # Project back: 3072 → 768
        x = self.dropout(x)      # Regularization
        return x

feed_forward = FeedForward(config)
ff_output = feed_forward(inputs_embeds)  # [1, 5, 768]

Visualizing Attention with BertViz:

from bertviz.transformers_neuron_view import BertModel
from bertviz.neuron_view import show
from transformers import AutoModel

model = AutoModel.from_pretrained(model_ckpt, output_attentions=True)

sentence_a = "time flies like an arrow"
sentence_b = "fruit flies like a banana"

viz_inputs = tokenizer(sentence_a, sentence_b, return_tensors='pt')
attention = model(**viz_inputs).attentions

# Visualize attention head 8 in first layer
from bertviz import head_view
sentence_b_start = (viz_inputs.token_type_ids == 0).sum(dim=1)
tokens = tokenizer.convert_ids_to_tokens(viz_inputs.input_ids[0])
head_view(attention, tokens, sentence_b_start, heads=[8])

Mathematical Concepts: - Matrix multiplications for batch attention computation - Softmax normalization:  $\exp(x) / \sum(\exp(x))$  - Dimensionality reduction via  $d_k$  prevents gradient vanishing - Typical config: hidden_size=768, num_heads=12, head_dim=64

Key Learnings: - Attention allows each token to interact with all others - Multiple heads capture different aspects of relationships - Feed-forward networks add non-linearity - Full understanding requires knowledge of PyTorch tensor operations - Transformer depth and width scale together

Technologies Used: PyTorch, Transformers, BertViz

```

Notebook 4: Multilingual Named Entity Recognition

File: 04_multilingual-ner.ipynb

Main Topics: - Named Entity Recognition (NER) tasks - Multilingual models (XLM-RoBERTa) - Tokenization differences across models - SentencePiece tokenizer - Token-level classification architecture

Key Sections:

1. The Dataset - PAN-X Corpus:

```
from datasets import load_dataset, get_dataset_config_names
import pandas as pd

# List available configurations
xtreme_subsets = get_dataset_config_names("xtreme")
print(f"XTREME has {len(xtreme_subsets)} configurations")

# Load German PAN-X dataset
ds_de = load_dataset("xtreme", name="PAN-X.de")

# Explore structure
print(ds_de["train"][0]) # Sample annotation

# Create multi-language dataset with proportional sampling
from collections import defaultdict
from datasets import DatasetDict

langs = ["de", "fr", "it", "en"]
fracs = [0.629, 0.229, 0.084, 0.059] # Language proportions
panx_ch = defaultdict(DatasetDict)

for lang, frac in zip(langs, fracs):
    ds = load_dataset("xtreme", name=f"PAN-X.{lang}")
    for split in ds:
        panx_ch[lang][split] = (
            ds[split]
            .shuffle(seed=0)
            .select(range(int(frac * ds[split].num_rows)))

# Show dataset statistics
pd.DataFrame({lang: [panx_ch[lang]["train"].num_rows] for lang in langs},
             index=["Number of training examples"])
```

Dataset Tags: - XTREME benchmark multilingual corpus - 4 languages: German (62.9%), French (22.9%), Italian (8.4%), English (5.9%) - NER tags: B-PER, I-PER, B-ORG, I-ORG, B-LOC, I-LOC, O (outside) - Sequence annotation format

2. Tokenization Comparison:

```
from transformers import AutoTokenizer

bert_model_name = "bert-base-cased"
xlmr_model_name = "xlm-roberta-base"
```

```

bert_tokenizer = AutoTokenizer.from_pretrained(bert_model_name)
xlmr_tokenizer = AutoTokenizer.from_pretrained(xlmr_model_name)

text = "Jack Sparrow loves New York!"
bert_tokens = bert_tokenizer(text).tokens()
xlmr_tokens = xlmr_tokenizer(text).tokens()

print("BERT tokens:", bert_tokens)
# ['Jack', 'Sparrow', 'loves', 'New', 'York', '!']

print("XLM-R tokens:", xlmr_tokens)
# ['Jack', 'Spar', 'row', 'loves', 'New', 'York', '!']

# SentencePiece uses  to represent spaces
print("".join(xlmr_tokens).replace(u"\u2581", " "))
# 'Jack Sparrow loves New York !'

```

3. Custom NER Model Architecture:

```

import torch.nn as nn
from transformers import XLMRobertaConfig, RobertaModel
from transformers.models.roberta.modeling_roberta import RobertaPreTrainedModel
from transformers.modeling_outputs import TokenClassifierOutput

class XLMRobertaForTokenClassification(RobertaPreTrainedModel):
    config_class = XLMRobertaConfig

    def __init__(self, config):
        super().__init__(config)
        self.num_labels = config.num_labels

        # Load model body (encoder)
        self.roberta = RobertaModel(config, add_pooling_layer=False)

        # Token classification head
        self.dropout = nn.Dropout(config.hidden_dropout_prob)
        self.classifier = nn.Linear(config.hidden_size, config.num_labels)

        # Initialize weights
        self.init_weights()

    def forward(self, input_ids=None, attention_mask=None, token_type_ids=None,
                labels=None, **kwargs):
        # Get encoder representations
        outputs = self.roberta(input_ids, attention_mask=attention_mask,
                               token_type_ids=token_type_ids, **kwargs)

```

```

# Apply dropout and classifier
sequence_output = self.dropout(outputs[0]) # [batch_size, seq_len, 768]
logits = self.classifier(sequence_output) # [batch_size, seq_len, num_labels]

# Compute loss if labels provided
loss = None
if labels is not None:
    loss_fct = nn.CrossEntropyLoss()
    loss = loss_fct(logits.view(-1, self.num_labels), labels.view(-1))

return TokenClassifierOutput(
    loss=loss, logits=logits, hidden_states=outputs.hidden_states)

```

4. Tag Conversion Example:

```

from collections import Counter

# Create human-readable tags
def create_tag_names(batch):
    tags = panx_ch["de"]["train"].features["ner_tags"].feature
    return {"ner_tags_str": [tags.int2str(idx) for idx in batch["ner_tags"]]}

panx_de = panx_ch["de"].map(create_tag_names)

# Show example with tags
de_example = panx_de["train"][0]
pd.DataFrame([de_example["tokens"], de_example["ner_tags_str"]],
              ['Tokens', 'Tags'])

# Analyze tag frequencies
split2freqs = defaultdict(Counter)
for split, dataset in panx_de.items():
    for row in dataset["ner_tags_str"]:
        for tag in row:
            if tag.startswith("B"): # Count only beginning tags
                tag_type = tag.split("-")[1]
                split2freqs[split][tag_type] += 1

pd.DataFrame.from_dict(split2freqs, orient="index")

```

Dataset Statistics: - German training: 12,552 examples - Multi-label handling with B/I tag scheme

Key Learnings: - Token boundaries vary significantly between tokenizers - SentencePiece handles subword units elegantly with symbol - NER requires token-level predictions, not document-level - Multilingual models use shared vocabulary

Technologies Used: Datasets, XLM-RoBERTa, SentencePiece, Transformers

Notebook 5: Text Generation

File: 05_text-generation.ipynb

Main Topics: - Autoregressive text generation strategies - Decoding methods: greedy, beam search, sampling - Temperature and probability truncation - Language model capabilities

Key Sections: 1. **The Challenge with Generating Coherent Text** - Exposure bias problem - Repetition issues - Hallucination challenges

2. Greedy Search Decoding

- Selecting highest probability token at each step
- Fast but often produces repetitive/poor quality text
- Implementation: `argmax()` over logits
- Example: “Transformers are the best... best... best...”

3. Beam Search Decoding

- Maintains k hypotheses (beams)
- Tracks sequence log-probabilities
- Prevents underflow with log-space computation
- Includes n-gram repetition penalty
- Typically uses k=5 beams

4. Sampling Methods

- **Temperature Scaling**
 - Lower T (0.5): more confident/focused
 - Higher T (2.0): more diverse/creative
 - Formula: $\text{softmax}(\text{logits}/T)$
- **Top-k Sampling**
 - Only sample from top k highest probability tokens
 - Common: k=50
- **Nucleus (Top-p) Sampling**
 - Sample from tokens with cumulative probability p
 - More flexible than top-k
 - Common: p=0.95

5. Model Used: GPT-2 XL

- Causal language model
- Generates continuations from prompts
- Pre-trained on diverse web text

Code Examples - Greedy Search:

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
import pandas as pd
```

```

device = "cuda" if torch.cuda.is_available() else "cpu"
model_name = "gpt2-xl"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name).to(device)

input_txt = "Transformers are the"
input_ids = tokenizer(input_txt, return_tensors="pt")["input_ids"].to(device)

```

```

# Greedy decoding
output_greedy = model.generate(input_ids, max_length=128, do_sample=False)
print(tokenizer.decode(output_greedy[0]))
# "Transformers are the most important..."

```

Code Examples - Beam Search:

```

# Beam search with 5 beams
output_beam = model.generate(input_ids, max_length=128, num_beams=5,
                             do_sample=False)
print(tokenizer.decode(output_beam[0]))

# Beam search with n-gram repetition penalty
output_beam_no_repeat = model.generate(input_ids, max_length=128, num_beams=5,
                                       do_sample=False, no_repeat_ngram_size=2)
print(tokenizer.decode(output_beam_no_repeat[0]))

```

Code Examples - Sampling Methods:

```

# Temperature scaling - Lower = more focused
output_temp_low = model.generate(input_ids, max_length=128, do_sample=True,
                                 temperature=0.5, top_k=0)
print(tokenizer.decode(output_temp_low[0]))

# Temperature scaling - Higher = more diverse
output_temp_high = model.generate(input_ids, max_length=128, do_sample=True,
                                  temperature=2.0, top_k=0)
print(tokenizer.decode(output_temp_high[0]))

# Top-k sampling
output_topk = model.generate(input_ids, max_length=128, do_sample=True,
                             top_k=50)

# Nucleus (top-p) sampling
output_nucleus = model.generate(input_ids, max_length=128, do_sample=True,
                                top_p=0.95, top_k=0)

```

Computing Sequence Log-Probability:

```

import torch.nn.functional as F

```

```

def log_probs_from_logits(logits, labels):
    logp = F.log_softmax(logits, dim=-1)
    logp_label = torch.gather(logp, 2, labels.unsqueeze(2)).squeeze(-1)
    return logp_label

def sequence_logprob(model, labels, input_len=0):
    """Compute log probability of a sequence"""
    with torch.no_grad():
        output = model(labels)
        log_probs = log_probs_from_logits(
            output.logits[:, :-1, :], labels[:, 1:])
        seq_log_prob = torch.sum(log_probs[:, input_len:])
    return seq_log_prob.cpu().numpy()

# Compare sequences
logp_greedy = sequence_logprob(model, output_greedy, input_len=len(input_ids[0]))
logp_beam = sequence_logprob(model, output_beam, input_len=len(input_ids[0]))
print(f"Greedy log-prob: {logp_greedy:.2f}")
print(f"Beam log-prob: {logp_beam:.2f}") # Beam search typically higher

```

Practical Example: - Generating continuations for: “In a shocking finding, scientist discovered a herd of unicorns...” - Comparing greedy vs beam search vs sampling outputs - Beam search produces more coherent text - Sampling with temperature enables creative variations

Key Learnings: - Greedy search is fast but low quality - Beam search balances quality and diversity - Sampling methods enable creative generations - Temperature controls diversity-confidence tradeoff - Log-probabilities prevent numerical underflow - Log-space computation prevents underflow: $\log(0.5^{1024}) \approx -710$

Technologies Used: Transformers, PyTorch, GPT-2

Notebook 6: Summarization

File: 06_summarization.ipynb

Main Topics: - Abstractive summarization - CNN/DailyMail dataset - Multiple pre-trained summarization models - BLEU and ROUGE evaluation metrics

Key Sections:

1. The CNN/DailyMail Dataset:

```

from datasets import load_dataset

# Load CNN/DailyMail dataset
dataset = load_dataset("cnn_dailymail", version="3.0.0")
print(f"Features: {dataset['train'].column_names}")

```

```

# Features: ['article', 'highlights', 'id']

# Explore sample
sample = dataset["train"][1]
print(f"Article length: {len(sample['article'])} chars")
print(f"Article excerpt: {sample['article'][:500]}")
print(f"\nSummary length: {len(sample['highlights'])} chars")
print(f"Summary: {sample['highlights']}")

```

- News articles with multi-sentence highlights (summaries)
- Train/validation/test splits (287K training examples)
- Features: article, highlights (summary)
- Typical article length: 500+ words
- Typical summary: 3-4 sentences

2. Text Summarization Pipelines:

```

from transformers import pipeline, set_seed
import pandas as pd
from nltk.tokenize import sent_tokenize

sample_text = dataset["train"][1]["article"][:2000]
summaries = {}

# Baseline: Extract first 3 sentences
def three_sentence_summary(text):
    return "\n".join(sent_tokenize(text)[:3])

summaries["baseline"] = three_sentence_summary(sample_text)

# GPT-2 with TL;DR prompt
set_seed(42)
pipe_gpt2 = pipeline("text-generation", model="gpt2-xl")
gpt2_query = sample_text + "\nTL;DR:\n"
pipe_out = pipe_gpt2(gpt2_query, max_length=512, clean_up_tokenization_spaces=True)
summaries["gpt2"] = "\n".join(sent_tokenize(pipe_out[0]["generated_text"][:len(gpt2_query)]))

# T5: Text-to-text framework
pipe_t5 = pipeline("summarization", model="t5-large")
pipe_out = pipe_t5(sample_text)
summaries["t5"] = "\n".join(sent_tokenize(pipe_out[0]["summary_text"]))

# BART: Pre-trained on CNN/DailyMail
pipe_bart = pipeline("summarization", model="facebook/bart-large-cnn")
pipe_out = pipe_bart(sample_text)
summaries["bart"] = "\n".join(sent_tokenize(pipe_out[0]["summary_text"]))

```

```

# PEGASUS: Domain-specific pre-training
pipe_pegasus = pipeline("summarization", model="google/pegasus-cnn_dailymail")
pipe_out = pipe_pegasus(sample_text)
summaries["pegasus"] = pipe_out[0]["summary_text"].replace(" .<n>", ".\n")

# Compare all summaries
for model_name in summaries:
    print(f"\n{model_name.upper()}:")
    print(summaries[model_name])
    print("=" * 50)

```

3. Model Comparison and Evaluation:

```

# BLEU Metric Example
from datasets import load_metric

bleu_metric = load_metric("sacrebleu")

# Poor example
bleu_metric.add(
    prediction="the the the the the the",
    reference=["the cat is on the mat"])
results = bleu_metric.compute(smooth_method="floor", smooth_value=0)
print(results["precisions"]) # [0.67, 0.0, 0.0, 0.0] - mostly 1-gram matches

# Better example
bleu_metric.add(
    prediction="the cat is on mat",
    reference=["the cat is on the mat"])
results = bleu_metric.compute(smooth_method="floor", smooth_value=0)
print(results["precisions"]) # [1.0, 0.8, 0.67, 0.5] - better n-gram coverage

```

ROUGE Metric Example:

```

rouge_metric = load_metric("rouge")

reference = dataset["train"][1]["highlights"]
records = []
rouge_names = ["rouge1", "rouge2", "rougeL", "rougeLsum"]

for model_name in summaries:
    rouge_metric.add(prediction=summaries[model_name], reference=reference)
    score = rouge_metric.compute()
    rouge_dict = dict((rn, score[rn].mid.fmeasure) for rn in rouge_names)
    records.append(rouge_dict)

results_df = pd.DataFrame.from_records(records, index=summaries.keys())
print(results_df) # Compare ROUGE scores across models

```

Typical results show PEGASUS > BART > T5 > baseline

Model Comparison - T5-large: Text-to-text universal model (versatile but generic) - facebook/BART-large-CNN: Pre-trained on CNN/DailyMail (specialized) - google/PEGASUS-cnn_dailymail: Domain-specific pre-training (highest quality) - Quality comparison on same input

BLEU (Bilingual Evaluation Understudy) - Precision-based n-gram matching - 1-gram, 2-gram, 3-gram, 4-gram precision - Brevity penalty: $BP = \exp(1 - \text{ref_len}/\text{hyp_len})$ - $BLEU = BP \times \exp(\log(p_n))$ where p_n is n-gram precision

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) - Recall-based n-gram overlap - ROUGE-1: unigram recall - ROUGE-2: bigram recall - ROUGE-L: longest common subsequence (handles word order) - ROUGE-Lsum: sentence-level LCS (better for multi-sentence summaries)

5. Evaluation Pipeline:

Full evaluation function

```
def evaluate_summaries(dataset, pipe, metric, model_name,
                       column_text="article", column_summary="highlights",
                       sample_size=100):
    """Evaluate summarization pipeline on dataset subset"""
    summaries = []

    for i in range(min(sample_size, len(dataset))):
        article = dataset[column_text][i]
        summary = pipe(article)
        summaries.append(summary[0]["summary_text"])

    metric.add_batch(predictions=summaries,
                     references=dataset[column_summary][:sample_size])
    score = metric.compute()

    return {
        "model": model_name,
        "rouge1": score["rouge1"].mid.fmeasure,
        "rouge2": score["rouge2"].mid.fmeasure,
        "rougeL": score["rougeL"].mid.fmeasure,
    }

# Evaluate all models
results = []
for model_name, pipe in [("t5", pipe_t5), ("bart", pipe_bart)]:
    result = evaluate_summaries(dataset["test"], pipe, rouge_metric, model_name)
    results.append(result)
```

```
comparison_df = pd.DataFrame(results)
print(comparison_df)  # Model comparison table
```

Key Learnings: - Different models have different strengths - Domain-specific pre-training (PEGASUS) often outperforms generic models - BLEU measures precision, ROUGE measures recall - Multiple reference summaries improve evaluation robustness - Abstractive summaries require semantic understanding

Technologies Used: Transformers, Datasets, sacrebleu, rouge

Notebook 7: Question Answering

File: 07_question_answering_v2.ipynb

Main Topics: - Extractive question answering - SQuAD and SubjQA datasets
- Span prediction for answer extraction - Handling long passages

Key Sections: 1. **Review-Based QA System** - SubjQA dataset: questions answerable from product reviews - Multiple domains: electronics, books, movies, etc. - Questions grounded in review text

2. The SubjQA Dataset

- Domain-specific (electronics used in examples)
- Format: question, context (review), answer span + position
- Answer start positions for exact span matching
- Multiple valid answers per question

3. Question Types Analysis

- What: 25%+ of questions
- How: Question decomposition
- Is/Does: Yes/no questions
- Where/Why: Less common

4. Extracting Answers via Span Classification

- Architecture: encoder → classification head
- Two logit outputs: start position, end position
- Training: token-level cross-entropy loss
- Inference: argmax over start/end logits

5. Tokenization for QA

- Question + context concatenation
- Special tokens: [CLS], [SEP], [PAD]
- Token IDs and attention masks
- Mapping back to original tokens

6. Model: deepset/minilm-uncased-squad2

- Fine-tuned on SQuAD 2.0
- Handles unanswerable questions
- Two output heads: start_logits, end_logits

7. Inference Process

- Get start/end token positions

- Decode to answer span
- Handle impossible answers
- Top-k predictions with confidence scores

8. Full Question Answering Pipeline:

```
from transformers import AutoTokenizer, AutoModelForQuestionAnswering, pipeline
import torch

model_ckpt = "deepset/minilm-uncased-squad2"
tokenizer = AutoTokenizer.from_pretrained(model_ckpt)
model = AutoModelForQuestionAnswering.from_pretrained(model_ckpt)

question = "How much music can this hold?"
context = """An MP3 is about 1 MB/minute, so about 6000 hours depending on file size."""

# Tokenize
inputs = tokenizer(question, context, return_tensors="pt")
print(tokenizer.decode(inputs["input_ids"][0]))
# [CLS] how much music can this hold? [SEP] An MP3 is about 1 MB / minute...[SEP]

# Get logits
with torch.no_grad():
    outputs = model(**inputs)

start_logits = outputs.start_logits # [1, seq_len]
end_logits = outputs.end_logits     # [1, seq_len]

# Find answer span
start_idx = torch.argmax(start_logits)
end_idx = torch.argmax(end_logits) + 1
answer_span = inputs["input_ids"][0][start_idx:end_idx]
answer = tokenizer.decode(answer_span)
print(f"Question: {question}")
print(f"Answer: {answer}") # "6000 hours"

# Using pipeline (simpler)
pipe = pipeline("question-answering", model=model, tokenizer=tokenizer)
results = pipe(question=question, context=context, topk=3)
for i, result in enumerate(results, 1):
    print(f"{i}. Answer: {result['answer']}, Score: {result['score']:.3f}")

# Handle impossible answers
impossible_result = pipe(question="Why is there no data?", context=context,
                        handle_impossible_answer=True)
print(impossible_result) # {'answer': '', 'score': 0.0}
```

8. Handling Long Passages:

```
def answer_long_context(question, long_context, tokenizer, model,
                        max_length=512, stride=128):
    """Handle contexts longer than model's max_length using sliding window"""
    answers = []

    # Split into overlapping windows
    tokens = tokenizer.encode(long_context)

    for i in range(0, len(tokens), stride):
        window = tokens[max(0, i - stride):i + max_length]

        # Get answer for this window
        window_text = tokenizer.decode(window)
        qa_inputs = tokenizer(question, window_text, return_tensors="pt")

        with torch.no_grad():
            outputs = model(**qa_inputs)

            start_idx = torch.argmax(outputs.start_logits).item()
            end_idx = torch.argmax(outputs.end_logits).item() + 1

            answer = tokenizer.decode(
                qa_inputs["input_ids"][0][start_idx:end_idx])
            answers.append(answer)

    return answers # Multiple candidates from different windows
```

Key Learnings: - QA requires precise span identification - Start/end logits are learned independently (not jointly constrained) - Context length is a practical limitation (typically 512 tokens) - Pipeline abstracts away tokenization complexity - Handle impossible answers for robust systems - Sliding window enables handling long documents

Technologies Used: Transformers, Datasets, SubjQA, SQuAD

Notebook 8: Making Transformers Efficient in Production

File: 08_model-compression.ipynb

Main Topics: - Production model optimization - Performance benchmarking - Knowledge distillation - Model compression techniques

Key Sections: 1. **Intent Detection Case Study** - CLINC intent detection dataset - 150 intent classes + out-of-scope - Real customer service queries - Baseline: BERT-based classifier

2. Performance Benchmarking Framework:

```
class PerformanceBenchmark:
    def __init__(self, pipeline, dataset, optim_type="BERT baseline"):
        self.pipeline = pipeline
        self.dataset = dataset
        self.optim_type = optim_type

    def compute_accuracy(self):
        pass    # Implemented below

    def compute_size(self):
        pass    # Implemented below

    def time_pipeline(self):
        pass    # Implemented below

    def run_benchmark(self):
        metrics = {}
        metrics[self.optim_type] = self.compute_size()
        metrics[self.optim_type].update(self.time_pipeline())
        metrics[self.optim_type].update(self.compute_accuracy())
        return metrics
```

- Creating custom PerformanceBenchmark class
- Three metrics:
 - **Size:** Model parameter count in MB
 - **Latency:** Inference time (average + std)
 - **Accuracy:** On test set

3. Measuring Model Size:

```
import torch
from pathlib import Path

def compute_size(pipeline):
    """Compute model size in MB"""
    state_dict = pipeline.model.state_dict()
    tmp_path = Path("model.pt")
    torch.save(state_dict, tmp_path)

    # Calculate size in megabytes
    size_mb = Path(tmp_path).stat().st_size / (1024 * 1024)

    # Delete temporary file
    tmp_path.unlink()
    print(f"Model size (MB) - {size_mb:.2f}")
    return {"size_mb": size_mb}
```

Example: BERT-base ~110MB, DistilBERT ~67MB (40% smaller)

4. Measuring Latency:

```
from time import perf_counter
import numpy as np

def time_pipeline(pipeline, query="What is the pin number for my account?"):
    """Measure inference latency with statistics"""
    latencies = []

    # Warmup runs (GPU cache warming)
    for _ in range(10):
        _ = pipeline(query)

    # Timed runs
    for _ in range(100):
        start_time = perf_counter()
        _ = pipeline(query)
        latency = perf_counter() - start_time
        latencies.append(latency)

    # Compute statistics
    time_avg_ms = 1000 * np.mean(latencies)
    time_std_ms = 1000 * np.std(latencies)

    print(f"Average latency (ms) - {time_avg_ms:.2f} +/- {time_std_ms:.2f}")
    return {"time_avg_ms": time_avg_ms, "time_std_ms": time_std_ms}

# Typical latencies:
# BERT-base: 25-40ms
# DistilBERT: 10-15ms (2.5x faster)
```

5. Computing Accuracy:

```
from datasets import load_metric, load_dataset

accuracy_metric = load_metric("accuracy")
clinc_data = load_dataset("clinc_oos", "plus")
intents = clinc_data["test"].features["intent"]

def compute_accuracy(pipeline, dataset):
    """Compute accuracy on test set"""
    preds, labels = [], []

    for example in dataset:
        pred = pipeline(example["text"])[0]["label"]
```

```

        label = example["intent"]
        preds.append(intents.str2int(pred))
        labels.append(label)

    accuracy = accuracy_metric.compute(predictions=preds, references=labels)
    print(f"Accuracy on test set - {accuracy['accuracy']:.3f}")
    return accuracy

result = compute_accuracy(pipeline, clinc_data["test"])
# Typical: BERT-base ~98.5%, DistilBERT ~97.8%

```

6. Knowledge Distillation Framework:

```

from transformers import TrainingArguments

class DistillationTrainingArguments(TrainingArguments):
    """Extended training arguments for knowledge distillation"""
    def __init__(self, *args, alpha=0.5, temperature=2.0, **kwargs):
        super().__init__(*args, **kwargs)
        self.alpha = alpha # Balance: 0.5 means equal weight
        self.temperature = temperature # Higher = softer targets

```

Distillation Loss Function:

```

import torch
import torch.nn as nn
import torch.nn.functional as F

def distillation_loss(student_logits, teacher_logits, labels,
                      temperature=2.0, alpha=0.5):
    """
    Combined loss = alpha * cross_entropy + (1-alpha) * distillation

    Cross-entropy: Hard targets (true labels)
    Distillation: Soft targets (teacher probabilities)
    """

    # Cross-entropy loss with hard targets
    ce_loss = F.cross_entropy(student_logits, labels)

    # KL divergence with soft targets (temperature-scaled)
    student_probs = F.log_softmax(student_logits / temperature, dim=-1)
    teacher_probs = F.softmax(teacher_logits / temperature, dim=-1)
    kl_loss = F.kl_div(student_probs, teacher_probs, reduction='batchmean')

    # Combined loss
    total_loss = alpha * ce_loss + (1 - alpha) * (temperature ** 2) * kl_loss
    return total_loss

```

Custom Distillation Trainer:

```
from transformers import Trainer

class DistillationTrainer(Trainer):
    def __init__(self, *args, teacher_model=None, **kwargs):
        super().__init__(*args, **kwargs)
        self.teacher_model = teacher_model

    def compute_loss(self, model, inputs, return_outputs=False):
        device = next(model.parameters()).device
        inputs = {k: v.to(device) for k, v in inputs.items()}

        # Get student outputs
        outputs_stu = model(**inputs)
        loss_ce = outputs_stu.loss
        student_logits = outputs_stu.logits

        # Get teacher outputs
        with torch.no_grad():
            outputs_tea = self.teacher_model(**inputs)
            teacher_logits = outputs_tea.logits

        # Compute distillation loss
        temperature = self.args.temperature
        alpha = self.args.alpha

        loss_kl = distillation_loss(
            student_logits, teacher_logits, inputs['labels'],
            temperature=temperature, alpha=alpha)

        return (loss_kl, outputs_stu) if return_outputs else loss_kl

# Usage
trainer = DistillationTrainer(
    model=student_model,
    args=DistillationTrainingArguments(..., alpha=0.5, temperature=2.0),
    teacher_model=teacher_model,
    train_dataset=train_dataset,
    compute_metrics=compute_metrics,
)
trainer.train()
```

Training Phases: - **Distillation for Fine-Tuning:** Student learns task-specific knowledge - **Distillation for Pre-training:** Student learns general language understanding

Results: - DistilBERT: ~40% smaller, ~60% faster, >99% accuracy retained - Perfect for mobile/edge deployment

Key Learnings: - Production requires multiple optimization dimensions - Knowledge distillation maintains quality while reducing size - Temperature controls knowledge transfer rate - Benchmarking is essential for production decisions

Technologies Used: Transformers, Datasets, Trainer API

Notebook 9: Dealing with Few to No Labels

File: 09_few-to-no-labels.ipynb

Main Topics: - Training with limited labeled data - Multi-label classification - Semi-supervised learning - GitHub issues dataset for labeling

Code Examples - Data Collection:

```
import math
import requests
from pathlib import Path
import pandas as pd
from tqdm.auto import tqdm
import time

def fetch_issues(owner="huggingface", repo="transformers",
                 num_issues=10_000, rate_limit=5_000):
    """Fetch GitHub issues with rate limiting"""
    batch = []
    all_issues = []
    per_page = 100 # API returns max 100 per page
    num_pages = math.ceil(num_issues / per_page)
    base_url = "https://api.github.com/repos"

    for page in tqdm(range(num_pages)):
        # Query with state=all to get both open and closed issues
        query = f"issues?page={page}&per_page={per_page}&state=all"
        issues = requests.get(f"{base_url}/{owner}/{repo}/{query}")
        batch.extend(issues.json())

        # Handle rate limiting
        if len(batch) > rate_limit and len(all_issues) < num_issues:
            all_issues.extend(batch)
            batch = []
            print(f"Reached GitHub rate limit. Sleeping for one hour...")
            time.sleep(60 * 60 + 1) # Sleep 1 hour
```

```

all_issues.extend(batch)

# Save to JSONL format (one JSON object per line)
df = pd.DataFrame.from_records(all_issues)
df.to_json(f"github-issues-{repo}.jsonl", orient="records", lines=True)
print(f"Saved {len(df)} issues")
return df

# Usage
# df_issues = fetch_issues()

Code Examples - Data Preparation:

import pandas as pd
import numpy as np
from sklearn.preprocessing import MultiLabelBinarizer
from skmultilearn.model_selection import iterative_train_test_split

# Load pre-downloaded issues
dataset_url = "https://git.io/nlp-with-transformers"
df_issues = pd.read_json(dataset_url, lines=True)
print(f"DataFrame shape: {df_issues.shape}") # (7,500, 30)

# Extract label names
df_issues["labels"] = (
    df_issues["labels"]
    .apply(lambda x: [meta["name"] for meta in x]))

# Filter to top 8 labels
label_map = {
    "Core: Tokenization": "tokenization",
    "New model": "new model",
    "Core: Modeling": "model training",
    "Usage": "usage",
    "Core: Pipeline": "pipeline",
    "TensorFlow": "tensorflow or tf",
    "PyTorch": "pytorch",
    "Examples": "examples",
    "Documentation": "documentation"
}

def filter_labels(x):
    return [label_map[label] for label in x if label in label_map]

df_issues["labels"] = df_issues["labels"].apply(filter_labels)
all_labels = list(label_map.values())

```

```

# Combine title and body
df_issues["text"] = (
    df_issues
    .apply(lambda x: x["title"] + "\n\n" + x["body"], axis=1))

# Remove duplicates
len_before = len(df_issues)
df_issues = df_issues.drop_duplicates(subset="text")
print(f"Removed {(len_before-len(df_issues))/len_before:.2%} duplicates")

# Analyze text length
df_issues["word_count"] = df_issues["text"].str.split().apply(len)
print(f"Mean words per issue: {df_issues['word_count'].mean():.0f}")
print(f"Max words per issue: {df_issues['word_count'].max():.0f}")

Code Examples - Creating Training Sets:

from datasets import Dataset, DatasetDict
from sklearn.model_selection import train_test_split

# Prepare data
df_clean = df_issues[["text", "labels", "split"]].reset_index(drop=True).copy()
df_unsup = df_clean.loc[df_clean["split"] == "unlabeled", ["text", "labels"]]
df_sup = df_clean.loc[df_clean["split"] == "labeled", ["text", "labels"]]

# Multi-label binarizer
mlb = MultiLabelBinarizer()
mlb.fit([all_labels])
print(f"Number of labels: {len(all_labels)}")

# Balanced split using iterative stratification
def balanced_split(df, test_size=0.5):
    ind = np.expand_dims(np.arange(len(df)), axis=1)
    labels = mlb.transform(df["labels"])
    ind_train, _, ind_test, _ = iterative_train_test_split(
        ind, labels, test_size)
    return df.iloc[ind_train[:, 0]], df.iloc[ind_test[:, 0]]

np.random.seed(0)
df_train, df_tmp = balanced_split(df_sup, test_size=0.5)
df_valid, df_test = balanced_split(df_tmp, test_size=0.5)

print(f"Train: {len(df_train)}, Valid: {len(df_valid)}, Test: {len(df_test)}")

# Create Hugging Face dataset
ds = DatasetDict({

```

```

    "train": Dataset.from_pandas(df_train.reset_index(drop=True)),
    "valid": Dataset.from_pandas(df_valid.reset_index(drop=True)),
    "test": Dataset.from_pandas(df_test.reset_index(drop=True)),
    "unsup": Dataset.from_pandas(df_unsup.reset_index(drop=True))
})

```

Code Examples - Creating Training Slices:

```

# Create slices for learning curve analysis
np.random.seed(0)
all_indices = np.expand_dims(list(range(len(ds["train"]))), axis=1)
indices_pool = all_indices
labels = mlb.transform(ds["train"]["labels"])
train_samples = [8, 16, 32, 64, 128] # Different training set sizes
train_slices = []
last_k = 0

for k in train_samples:
    # Stratified sampling
    ind_train, indices_pool, _, labels = iterative_train_test_split(
        indices_pool, labels, test_size=(len(indices_pool) - k) / len(indices_pool))

    train_slices.append(ind_train[:, 0])
    print(f"Training slice {k}: selected indices")

# Use slices to create datasets
ds_slices = {
    k: Dataset.from_pandas(
        df_train.iloc[indices].reset_index(drop=True))
    for k, indices in zip(train_samples, train_slices)
}

print(f"Created {len(ds_slices)} training slices")

```

Dataset Statistics: - ~13% of issues have labels (~1,000) - ~87% unlabeled pool for semi-supervised learning (~6,500) - Total issues: ~7,500 - Top labels: tokenization, pytorch, documentation, model training

Key Learnings: - Most real-world data is unlabeled - Multi-label classification requires careful handling (each example has multiple labels) - Balanced sampling is important with limited labels - Learning curves show how much labeled data is needed - Iterative stratification preserves label distribution in splits

Technologies Used: Datasets, Pandas, scikit-multilearn

Notebook 10: Training Transformers from Scratch

File: 10_transformers-from-scratch.ipynb

Main Topics: - Building large-scale datasets - Pre-training language models from scratch - Custom tokenizer development - Working with massive corpora (100GB+) - Streaming and memory-mapped datasets

Code Examples - Model Scaling Comparison:

```
from transformers import pipeline, set_seed
import torch

# Compare GPT vs GPT-2 sizes
generation_gpt = pipeline("text-generation", model="openai-gpt")
generation_gpt2 = pipeline("text-generation", model="gpt2")

def model_size(model):
    return sum(t.numel() for t in model.parameters())

print(f"GPT size: {model_size(generation_gpt.model)/1000**2:.1f}M parameters")
# GPT size: 110.1M parameters
print(f"GPT2 size: {model_size(generation_gpt2.model)/1000**2:.1f}M parameters")
# GPT2 size: 124.4M parameters

# Generate and compare
set_seed(1)
prompt = "\nWhen they came back"

for name, pipe in [("GPT", generation_gpt), ("GPT-2", generation_gpt2)]:
    output = pipe(prompt, num_return_sequences=1, max_length=50)
    print(f"{name}: {output[0]['generated_text']}")
```

Code Examples - Loading Large Datasets:

```
from datasets import load_dataset, DownloadConfig
import psutil
import os

# Method 1: Memory mapping (load but don't cache)
print("Memory Mapping:")
dataset = load_dataset("./codeparrot", split="train")
print(f"Number of python files: {len(dataset)}")
ds_size = sum(os.stat(f["filename"]).st_size for f in dataset.cache_files)
print(f"Dataset size (cache): {ds_size / 2**30:.2f} GB")
mem_usage = psutil.Process(os.getpid()).memory_info().rss >> 20
print(f"RAM used: {mem_usage} MB") # Still manageable despite large dataset
```

```

# Method 2: Streaming (iterate without downloading)
print("\nStreaming:")
streamed_dataset = load_dataset('./codeparrot', split="train", streaming=True)
iterator = iter(streamed_dataset)

# Iterate through examples without loading entire dataset
for i, example in enumerate(iterator):
    if i < 5:
        print(f"Example {i}: {example['code'][:100]}...")
    if i == 100: # Just check first 100
        break

# Method 3: Remote streaming (even better!)
remote_dataset = load_dataset('transformersbook/codeparrot',
                               split="train", streaming=True)

```

Code Examples - Building Custom Tokenizer:

```

from transformers import AutoTokenizer

def tok_list(tokenizer, string):
    """Convert string to tokens"""
    input_ids = tokenizer(string, add_special_tokens=False)["input_ids"]
    return [tokenizer.decode(tok) for tok in input_ids]

# Load existing tokenizers
tokenizer_T5 = AutoTokenizer.from_pretrained("t5-base")
tokenizer_camembert = AutoTokenizer.from_pretrained("camembert-base")

# Compare tokenization
print(f'T5 tokens for "sex": {tok_list(tokenizer_T5, "sex")}')
# ['se', 'x']
print(f'CamemBERT tokens for "being": {tok_list(tokenizer_camembert, "being")}')
# ['being']

# GPT-2 tokenizer
tokenizer_gpt2 = AutoTokenizer.from_pretrained("gpt2")
python_code = r"""def say_hello():
    print("Hello, World!")
say_hello()"""

print("GPT-2 Python tokens:")
print(tokenizer_gpt2(python_code).tokens())

```

Code Examples - UTF-8 to Unicode Mapping for BPE:

```

from transformers.models.gpt2.tokenization_gpt2 import bytes_to_unicode

```

```

# Get byte-to-unicode mapping used in BPE
byte_to_unicode_map = bytes_to_unicode()
unicode_to_byte_map = dict((v, k) for k, v in byte_to_unicode_map.items())
base_vocab = list(unicode_to_byte_map.keys())

print(f'Size of base vocabulary: {len(base_vocab)}')
# Size of base vocabulary: 256
print(f'First element: `{base_vocab[0]}`, last element: `{base_vocab[-1]}`')

# Examples of mappings
a_byte = ord("a".encode("utf-8"))
print(f'`a` is encoded as `{ "a".encode("utf-8") }` -> {byte_to_unicode_map[a_byte]}')

euro_bytes = [ord(chr(i)) for i in "€".encode("utf-8")]
print(f'`€` is encoded as bytes: {euro_bytes}')
for byte in euro_bytes:
    print(f' Byte {byte} -> {byte_to_unicode_map[byte]}')

# After BPE, common subword units are merged
print(f"\nFinal tokenizer vocabulary size: {len(tokenizer_gpt2)}")
# Final tokenizer vocabulary size: 50257

```

Code Examples - Tokenizer Performance Measurement:

```

import time

def measure_compression(tokenizer, text):
    """Measure compression ratio (text/tokens)"""
    tokens = tokenizer.encode(text)
    compression = len(text) / len(tokens)
    return compression

# Compare compression ratios
code_sample = '''def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)'''

# Different tokenizers compress differently
print(f"GPT-2 compression: {measure_compression(tokenizer_gpt2, code_sample):.2f}")
# GPT-2 compression: 3.45 (general purpose)

# Domain-specific tokenizers would compress better for code:
# BPE trained on Python code might achieve 4.2 (fewer tokens needed)

```

Code Examples - Pre-training Setup:

```

from torch.utils.data import DataLoader

```

```

from transformers import DataCollatorForLanguageModeling

# Assuming tokenized dataset
tokenized_dataset = dataset.map(
    lambda x: tokenizer(x["code"], truncation=True, max_length=512),
    batched=True,
    remove_columns=["code"]
)

# Prepare data collator for masked language modeling
data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm_probability=0.15 # Mask 15% of tokens
)

# Create data loader
dataloader = DataLoader(
    tokenized_dataset,
    batch_size=32,
    collate_fn=data_collator
)

print(f"Number of batches: {len(dataloader)}")

```

Key Sections: 1. **Large Datasets and Challenges** - Data quality, filtering, deduplication 2. **Building Custom Code Dataset** - Google BigQuery filtering 3. **Memory Mapping** - Loading without storing entire dataset in RAM 4. **Streaming** - Infinite datasets, remote data sources 5. **Building Tokenizers** - BPE algorithm, vocabulary creation 6. **Tokenizer Pipeline** - Normalization, pre-tokenization, merging 7. **Domain-Specific Tokenizers** - Python code example 8. **UTF-8 Encoding** - Byte handling in BPE 9. **Tokenizer Performance** - Compression ratios, efficiency metrics

Dataset Scale Examples: - CodeParrot: ~180GB of Python code - Streaming: process without downloading - Memory-mapping: 2-3GB RAM for 180GB dataset

Key Learnings: - Pre-training requires careful data curation - Streaming enables training on datasets larger than RAM - Custom tokenizers improve compression for domain-specific text - UTF-8 encoding requires special handling for BPE - Token efficiency varies by domain (code vs natural language) - BPE merging is learned from data frequency

Technologies Used: Datasets, Tokenizers, BigQuery, PyTorch, Hugging Face Hub

Notebook 11: Future Directions

File: 11_future-directions.ipynb

Main Topics: - Scaling transformers to trillion parameters - Vision and multi-modal transformers - Efficient attention mechanisms - Future applications

Key Sections: 1. **Scaling Transformers**

Model Evolution: - Transformer (2017): 213M - GPT (2018): 110M - BERT (2018): 340M - GPT-2 (2019): 1.5B - T5 (2019): 11B - GPT-3 (2020): 175B - Switch-C (2021): 1.6T

Trend: Exponential growth following Moore's Law variant

2. Scaling Laws

- Power-law relationships discovered by OpenAI/DeepMind
- Test loss $\propto 1/C^\alpha$ (compute budget)
- Test loss $\propto 1/N^\beta$ (dataset size)
- Test loss $\propto 1/D^\gamma$ (model size)
- Cross-modal scaling laws (text, image, audio)

3. Challenges with Scaling

- Memory requirements
- Computational cost
- Training time
- Environmental impact
- Distributed training complexity

4. Attention Mechanisms at Scale

Sparse Attention Patterns:

- Standard attention: $O(n^2)$ complexity
- Strided attention
- Fixed attention patterns
- Learned sparse patterns
- Examples from Longformer, BigBird

Atomic Sparsity:

- Column attention (attending to full columns)
- Row attention (attending to specific rows)
- Combinations for efficiency

Linearized Attention:

- Approximating softmax with other functions
- Linear complexity $O(n)$
- Trade-off: reduced expressiveness

5. Going Beyond Text

Vision:

- iGPT: Image generation with transformers
- ViT (Vision Transformer): Image classification
- Patch-based tokenization
- CLIP: Vision-language alignment

Tables:

- TAPAS: Question-answering over tables
- Row/column attention
- Semantic similarity for cell matching

6. Multimodal Transformers

Speech-to-Text:

- wav2vec 2.0: Self-supervised speech learning
- CTC loss for alignment
- Contrastive learning

Vision-Language:

- CLIP: Image-text alignment
- Cross-modal retrieval
- Zero-shot classification

7. Applications Explored

- Image classification on “doge” example image
- Table question-answering on book chapter data
- Multi-domain applications

Code Examples - Model Scaling History:

```
import pandas as pd
import matplotlib.pyplot as plt

# Historical model parameter counts
model_data = [
    {'date': '2017-06-12', 'name': 'Transformer', 'size': 213e6},
    {'date': '2018-06-11', 'name': 'GPT', 'size': 110e6},
    {'date': '2018-10-11', 'name': 'BERT', 'size': 340e6},
    {'date': '2019-02-14', 'name': 'GPT-2', 'size': 1.5e9},
    {'date': '2019-10-23', 'name': 'T5', 'size': 11e9},
    {'date': '2020-02-17', 'name': 'Turing-NLG', 'size': 17e9},
    {'date': '2020-05-28', 'name': 'GPT-3', 'size': 175e9},
    {'date': '2021-01-11', 'name': 'Switch-C', 'size': 1.6e12},
]
```

```

df_lm = pd.DataFrame.from_records(model_data)
df_lm['date'] = pd.to_datetime(df_lm['date'])

# Plot on log scale
fig, ax = plt.subplots(figsize=(12, 6))
df_lm.plot(x='date', y='size', kind='scatter', s=100, ax=ax, logy=True)
for idx, row in df_lm.iterrows():
    ax.annotate(row['name'],
                (row['date'], row['size']),
                xytext=(5, 5), textcoords='offset points')
ax.set_ylabel('Number of Parameters (log scale)')
ax.set_xlabel('Release Date')
ax.set_title('Transformer Model Scaling Over Time')
plt.show()

print(f"Growth factor from Transformer to Switch-C:")
print(f"{model_data[-1]['size'] / model_data[0]['size']:.0f}x increase")
# ~7,500x increase in 4 years!

```

Code Examples - Scaling Laws:

```

import numpy as np

# Chinchilla scaling laws (DeepMind)
def compute_loss(compute_budget, alpha=0.62, beta=468.8):
    """
    Predicted loss based on compute budget
    Loss = alpha / C^0.051 + beta / C^0.05
    """
    c_exponent = 0.051
    return alpha / (compute_budget ** c_exponent) + beta / (compute_budget ** c_exponent)

# Optimal model/data allocation
compute = np.logspace(0, 4, 50)
loss = compute_loss(compute)

fig, ax = plt.subplots()
ax.loglog(compute, loss, linewidth=2, label='Predicted Loss')
ax.set_xlabel('Compute Budget (petaflops-days)')
ax.set_ylabel('Validation Loss')
ax.set_title('Chinchilla Scaling Laws')
ax.grid(True, alpha=0.3)
plt.show()

print("Key finding: Compute budget should be split as:")
print("- Model size: ~20% of compute")
print("- Training data: ~80% of compute")

```

```
print("This contradicts earlier thinking that favored larger models.")
```

Code Examples - Vision Transformers:

```
from PIL import Image
import matplotlib.pyplot as plt
from transformers import pipeline
import pandas as pd

# Load and display image
image = Image.open("images/doge.jpg")
plt.imshow(image)
plt.axis("off")
plt.show()

# Classify image with Vision Transformer
image_classifier = pipeline("image-classification",
                             model="google/vit-base-patch16-224")
preds = image_classifier(image)

# Display results
preds_df = pd.DataFrame(preds)
print(preds_df)
# Shows classes like 'Shiba Inu', 'Doge' with confidence scores

print("\nViT Architecture:")
print("1. Divide image into 16x16 patches (224x224 → 196 patches)")
print("2. Linear projection of each patch → embeddings")
print("3. Add class token and position embeddings")
print("4. Apply transformer encoder")
print("5. Use [CLS] token output for classification")
```

Code Examples - Table Question Answering:

```
from transformers import pipeline
import pandas as pd

# Create sample table
book_data = [
    {"chapter": 0, "name": "Introduction", "start_page": 1, "end_page": 11},
    {"chapter": 1, "name": "Text Classification", "start_page": 12, "end_page": 48},
    {"chapter": 2, "name": "NER", "start_page": 49, "end_page": 73},
    {"chapter": 3, "name": "Question Answering", "start_page": 74, "end_page": 120},
    {"chapter": 4, "name": "Summarization", "start_page": 121, "end_page": 140},
]

table = pd.DataFrame(book_data)
table['num_pages'] = table['end_page'] - table['start_page']
```

```

table = table.astype(str)

print("Sample Table:")
print(table)

# Table QA pipeline
table_qa = pipeline("table-question-answering")

queries = [
    "What's the topic in chapter 4?",
    "What is the total number of pages?",
    "On which page does chapter 3 start?",
    "How many chapters have more than 20 pages?"
]

preds = table_qa(table, queries)

print("\nTable QA Results:")
for query, pred in zip(queries, preds):
    print(f"Q: {query}")
    if pred["aggregator"] == "NONE":
        print(f"A: {pred['answer']}")
    else:
        print(f"A: {pred['answer']} (aggregated via {pred['aggregator']})")
    print()

```

Code Examples - Multimodal Speech-to-Text:

```

from transformers import pipeline
import librosa
import numpy as np

# Automatic speech recognition with wav2vec2
asr = pipeline("automatic-speech-recognition",
               model="facebook/wav2vec2-base-960h")

# Load audio file
audio_path = "sample_audio.wav"
audio, sr = librosa.load(audio_path, sr=16000)

# Transcribe
result = asr(audio)
print(f"Transcription: {result['text']}")
# Output: "hello how are you doing today"

print("\nwav2vec 2.0 Architecture:")
print("1. Continuous speech input (audio waveform)")

```

```

print("2. CNN feature extractor → acoustic features")
print("3. Quantization → discrete representation")
print("4. Transformer encoder with masked prediction")
print("5. CTC loss for alignment to text")
print("\nKey innovation: Self-supervised learning on raw audio")
print("- Predict masked frames from context")
print("- Learns linguistic features without transcriptions")

```

Code Examples - Sparse Attention Patterns:

```

print("Attention Sparsity Patterns:\n")

print("1. Standard Self-Attention ( $O(n^2)$ ):")
print("    Every token attends to all other tokens")
print("    Pattern: Full  $n \times n$  matrix\n")

print("2. Strided Attention:")
print("    Token  $i$  attends to tokens  $[i-s, i-2s, \dots, i, \dots, i+2s, i+s]$ ")
print("    Pattern: Diagonal stripes\n")

print("3. Fixed Attention:")
print("    Token  $i$  attends to positions  $[0, 1, \dots, w, n-w, \dots, n-1]$ ")
print("    Pattern: Local window + boundary\n")

print("4. Learned Sparse Patterns:")
print("    Model learns which positions to attend to")
print("    Pattern: Data-dependent sparsity\n")

print("Complexity reduction:")
for n in [512, 2048, 4096]:
    dense = n**2
    sparse = n * 64 # Approximate for sparse
    speedup = dense / sparse
    print(f"    Length {n}: {speedup:.1f}x speedup")

```

Code Examples - Linearized Attention:

```

print("Linearized Attention vs Standard Attention:\n")

print("Standard Attention ( $O(n^2)$  time complexity):")
print("    Scores = softmax( $Q @ K^T / \sqrt{d}$ )")
print("    Attention = Scores @ V")
print("    Output shape: [batch, n, d]\n")

print("Linearized Attention ( $O(n)$  time complexity):")
print("    Replace softmax with feature map ")
print("    Scores = (Q) @ (K)^T # Can be reordered!")
print("    Attention = ((Q) @ ((K)^T @ V)) # Compute K^T @ V first!")

```

```

print(" This reduces from  $O(n^2)$  to  $O(n)$  space!\n")

print("Trade-offs:")
print(" - Pro: Handles sequences > 100k tokens")
print(" - Con: Approximation, loses some expressiveness")
print(" - Use case: Long document processing")

```

Key Sections: 1. **Scaling Transformers** - Exponential growth, timeline of models 2. **Scaling Laws** - Power-law relationships (Chinchilla, OpenAI research) 3. **Challenges with Scaling** - Memory, compute, training time 4. **Attention at Scale** - Sparse and linearized attention mechanisms 5. **Vision Transformers** - Images divided into patches, then transformer 6. **Table Question Answering** - TAPAS for structured data 7. **Multimodal Learning** - Speech (wav2vec), vision (ViT), cross-modal (CLIP)

Model Scaling Timeline: - Transformer (2017): 213M parameters - GPT (2018): 110M $\rightarrow$ GPT-2: 1.5B $\rightarrow$ GPT-3: 175B - BERT (2018): 340M - T5 (2019): 11B - Switch-C (2021): 1.6 Trillion

Key Learnings: - Scaling follows predictable power-laws ($\text{loss} \propto C^{-0.051}$) - Attention is the computational bottleneck for long sequences - Sparse attention enables longer contexts (4k to 32k+ tokens) - Linearized attention reduces $O(n^2)$ to $O(n)$ complexity - Multimodal learning is the future direction - Efficient architectures needed for deployment - Vision transformers competitive with CNNs on image tasks - Transformer architecture generalizes beyond text to audio, images, tables

Technologies Used: Transformers, Vision models, CLIP, wav2vec 2.0, Librosa

Cross-Notebook Themes

Core Concepts

1. **Transformer Architecture** (Notebook 3)
 - Self-attention mechanisms
 - Multi-headed attention
 - Feed-forward networks
2. **Pre-trained Models** (All notebooks)
 - BERT family
 - GPT family
 - Domain-specific variants
3. **Tokenization** (Notebooks 4, 10)
 - WordPiece (BERT)
 - SentencePiece (multilingual)
 - Custom tokenizers
 - BPE algorithm
4. **Fine-tuning** (Notebooks 2, 4, 7, 8)

- Task-specific heads
- Classification layers
- Token-level predictions
- Span extraction

Practical Tools

- **Hugging Face Transformers:** Model loading, inference, training
- **Hugging Face Datasets:** Data loading, exploration, splitting
- **Transformers Hub:** Pre-trained models repository
- **Trainer API:** High-level training abstraction

Datasets Used

Notebook	Dataset	Task	Size
1	Text examples	Multi-task demo	Small
2	emotion	Sentiment/emotion classification	16K
4	xtreme (PAN-X)	Multilingual NER	~50K
6	cnn_dailymail	Summarization	287K
7	subjqa	Question answering	12K+
8	clinc_oos	Intent detection	22K+
9	github-issues	Multi-label tagging	7.5K

Key Takeaways

1. **Transformers are Versatile**
 - Same architecture for diverse tasks
 - Fine-tuning adapts to specific domains
 - Transfer learning reduces data requirements
2. **Practical Considerations**
 - Model size matters for production
 - Knowledge distillation enables efficiency
 - Benchmarking is essential
 - Data quality > data quantity
3. **Scale and Efficiency**
 - Larger models generally perform better
 - Scaling laws predict improvements
 - Sparse attention enables scaling
 - Efficient attention is critical frontier
4. **Multilingual and Multimodal**
 - Transformers extend beyond text
 - Shared representations across modalities
 - Vision transformers competitive with CNNs
 - Speech recognition with wave2vec2
5. **Limited Label Scenarios**

- Few-shot learning strategies
 - Semi-supervised approaches
 - Data augmentation techniques
 - Practical for real-world applications
-

Technology Stack Summary

Core Libraries: - PyTorch (deep learning framework) - Transformers (Hugging Face) - Datasets (Hugging Face) - Tokenizers (Hugging Face)

Supporting Libraries: - NumPy, Pandas (data manipulation) - Matplotlib, Plotly (visualization) - scikit-learn (ML utilities) - sacrebleu, rouge (evaluation)

Models Referenced: - BERT, DistilBERT - RoBERTa, XLM-RoBERTa - GPT, GPT-2 - T5, BART, PEGASUS - ViT (Vision Transformer) - CLIP, wav2vec 2.0

Learning Path Recommendations

For Beginners:

1. Start with Notebook 1 (Introduction) for concepts
2. Notebook 3 (Anatomy) for architecture understanding
3. Notebook 2 (Classification) for first practical task
4. Notebook 5 (Generation) for understanding decoding

For Intermediate Learners:

1. Notebook 4 (Multilingual NER) for tokenization depth
2. Notebook 6 (Summarization) for seq2seq tasks
3. Notebook 7 (QA) for span extraction
4. Notebook 8 (Compression) for production concerns

For Advanced Users:

1. Notebook 9 (Few-labels) for realistic scenarios
 2. Notebook 10 (From scratch) for pre-training
 3. Notebook 11 (Future) for cutting-edge directions
-

Conclusion

This notebook collection represents a complete curriculum for NLP with transformers, from foundational concepts to production deployment and research frontiers. The progression is pedagogically sound, with each notebook building

on previous knowledge while introducing new concepts and practical techniques. The emphasis on hands-on implementation through Hugging Face tools makes these notebooks valuable for both learning and reference.

The notebooks balance theoretical understanding (architecture, attention mechanisms) with practical application (classification, QA, summarization), and address real-world concerns (efficiency, limited labels, production optimization). Together, they provide a comprehensive resource for anyone working with transformer-based NLP systems.

Generated Report: NLP with Transformers - Complete Notebook Documentation